

Requirement Analysis

AI-Based Smart Book Recommendation System

Group 30

Member ID	Member Name
IN26011213	Devansh Phougat
IN26011778	Khushi Kumar
IN26012273	Kartikey Verma
IN26012176	Faiza Farzana
IN26011120	Krishnendra Pratap
IN26012341	Aman Dadhich
IN26011545	Utsav Raj

Understanding what needs to be built before deciding how to build it.

1. Position of Requirement Analysis in the Project Lifecycle

Requirement analysis is the first major activity in the project planning and design phase. It creates the foundation for system design, database design, AI model planning, sprint planning, testing, and development.

1	2	3	4	5	6
Business Problem	Requirement Analysis	System Design	Database Design	AI Model Design	Agile Sprint Planning

Key principle: Good software development starts with understanding the book-discovery problem before writing the recommendation solution.

2. What is Requirement Analysis?

Requirement Analysis is the process of identifying, understanding, documenting, and validating what users, administrators, and the organization need from an AI-based book recommendation system.

It helps the development team understand the expected functionality, users and roles, recommendation behavior, data requirements, AI expectations, security, performance, and conditions for accepting the finished system.

3. Why Requirement Analysis is Important

A customer statement such as “We need an application that recommends books to users” is a useful starting point, but it is not detailed enough for development.

The team needs answers to questions such as: Who can use the system? What information is required to create a profile? How are books searched and rated? How will recommendations be generated? What happens when a new user has no history? Can an administrator manage books and users? How is recommendation quality evaluated?

Requirement analysis turns these questions into clear, documented and testable requirements.

4. Case Study – AI-Based Smart Book Recommendation System

Many readers find it difficult to discover books that match their interests because a large catalogue contains thousands of titles. A basic library or online catalogue can search and display books, but it may not provide personalized recommendations.

Business Goal: Build a centralized smart book discovery system that allows users to search, rate, save and track books while an AI recommendation engine suggests books based on user preferences, reading history, ratings and book content.

At this stage, the focus is on understanding the business problem and required behavior—not choosing a specific programming language, framework, database or AI library.

Core AI idea: Use content-based recommendation as the initial model and allow a future hybrid model that combines content similarity with collaborative user behavior.

5. Stakeholders

A stakeholder is a person, group or organization that is interested in the system or affected by it.

Stakeholder	Interest / Need
Readers / Students	Discover relevant books, manage reading lists, rate books and receive personalized recommendations.
Librarians / Content Managers	Maintain book records, categories, authors and catalogue quality.
Administrators	Manage users, roles, books, system configurations and access control.
Project / Product Team	Monitor usage, recommendation quality, defects and system performance.
Institution / Organization	Provide a useful, secure and reliable digital book-discovery service.

6. Actors

An actor is a user or external system that directly interacts with the application.

Primary actors:

- User / Reader
- Librarian / Content Manager
- Administrator
- Recommendation Engine / AI Service

Important distinction: The AI recommendation engine is a system component or service. It is not a human user actor, but it participates in system behavior and decision support.

7. Functional Requirements

A functional requirement describes what the system should do.

Actor	Functional Requirements
User / Reader	Register and log in; manage profile; search and filter books; view book details; rate books; add books to wishlist/reading list; mark books as read; view personalized recommendations.
Recommendation Engine	Generate book recommendations; calculate similarity; use user preferences and reading history; produce recommendation scores; provide recommendation explanations; update recommendations after new feedback.
Librarian / Content Manager	Add, edit and remove book records; manage authors and genres; maintain descriptions and metadata; review catalogue quality.
Administrator	Manage users and roles; manage access; monitor system activity; view reports; configure recommendation settings; manage flagged or invalid data.
System	Store user activity; maintain recommendation history; record ratings; maintain audit logs; notify users of relevant system events.

AI-specific functional requirements:

- FR-AI-01: The system shall represent book descriptions and selected metadata as features for recommendation.
- FR-AI-02: The system shall calculate similarity between books using an approved recommendation method.
- FR-AI-03: The system shall generate a ranked list of recommendations for a user.
- FR-AI-04: The system shall use explicit feedback such as ratings and implicit feedback such as reading/wishlist activity when available.
- FR-AI-05: The system shall handle cold-start users by using selected interests, popular books, genres or onboarding preferences.

- FR-AI-06: The system shall provide an understandable reason for each recommendation where possible.
- FR-AI-07: The system shall allow recommendation quality to be evaluated using defined metrics and test data.

8. Non-Functional Requirements

A non-functional requirement describes how the system should behave or the constraints under which it should operate.

Examples:

- Response time
- Security
- Availability
- Scalability
- Reliability
- Usability
- Privacy
- AI recommendation quality
- Maintainability

9. Functional vs Non-Functional Requirements

Functional – What?	Non-Functional – How well / under what constraints?
Create a user profile	Secure authentication and privacy
Search for books	Search results should respond quickly
Rate a book	Ratings should be stored reliably
Generate recommendations	Recommendation response should meet defined performance targets
Explain a recommendation	Explanation should be understandable and consistent
Manage books	Only authorized users may modify catalogue data
Generate reports	System should remain available and scalable as users increase

10. Quick Classification Examples

“The user should be able to rate a book.” → Functional

“The recommendation page should load within 2 seconds under normal load.” → Non-functional

“The administrator should be able to deactivate a user.” → Functional

“Only authorized administrators should be able to manage users.” → Non-functional

“The system should recommend books based on the user's interests.” → Functional

“User reading history and ratings should be protected from unauthorized access.” → Non-functional

11. User Stories

In Agile development, requirements are often represented as user stories. A common format is:

As a [user], I want [functionality], so that [benefit].

This format identifies who needs the feature, what they need and why it provides value.

Example User Stories

- User: As a reader, I want to create a profile so that the system can personalize my recommendations.
- User: As a reader, I want to search for books so that I can quickly find a specific title or topic.
- User: As a reader, I want to rate books so that the system can learn my preferences.
- User: As a reader, I want to receive personalized recommendations so that I can discover books I am likely to enjoy.
- User: As a reader, I want to know why a book was recommended so that I can understand the recommendation.
- User: As a reader, I want to save books to a reading list so that I can return to them later.
- User: As a reader, I want to mark books as read so that my reading history remains accurate.
- Librarian: As a content manager, I want to update book metadata so that recommendations are based on accurate information.
- Administrator: As an administrator, I want to manage users and roles so that system access remains controlled.
- Administrator: As an administrator, I want to view recommendation and usage reports so that I can monitor system performance.

12. Acceptance Criteria

Acceptance criteria define the conditions that must be satisfied for a user story to be considered complete and acceptable.

Example User Story: As a reader, I want personalized book recommendations so that I can discover books relevant to my interests.

#	Acceptance Criterion
1	The user must be logged in to receive personalized recommendations.
2	The user profile must contain at least one preference, rating, reading-history item, or onboarding interest when available.
3	The system must retrieve eligible books from the catalogue.
4	The recommendation engine must calculate relevance or similarity using the configured model.
5	The system must return a ranked list of recommendations.
6	Books already marked as read may be excluded or handled according to configured rules.
7	Each recommendation should show an understandable reason when explanation data is available.
8	The user must be able to provide feedback that can influence future recommendations.

13. Given – When – Then

Acceptance criteria can also be written using a simple Given–When–Then structure.

- ✓ Given the user is logged in and has selected interests or previous book activity
- ✓ When the user opens the recommendation page
- ✓ Then the system should generate and display a ranked list of relevant books
- ✓ And each recommendation should show its title, author and a relevance explanation where available
- ✓ And the user should be able to rate or save a recommended book

14. From Requirement to Development

Requirement analysis creates the foundation for the work that follows. Each requirement should be traceable through system design, database work, AI model work, development, testing and sprint tasks.

Stage	Example
Business Requirement	Users need personalized book recommendations to reduce the difficulty of discovering relevant books.
User Story	As a reader, I want personalized recommendations so that I can discover books suited to my interests.
Acceptance Criteria	User profile/activity available; recommendations generated; ranked results displayed; explanation shown where possible.
System Design	User interface, authentication, book catalogue service, recommendation API, admin module and reporting.
Database Design	User, Book, Author, Genre, Rating, ReadingHistory, Wishlist, Recommendation and RecommendationFeedback entities.
AI Model Design	Text preprocessing, feature extraction, TF-IDF/content vectors, cosine similarity and optional collaborative filtering.
Development Task	Implement recommendation endpoint, user preference handling, ranking, explanation and feedback capture.
Testing Task	Test functional behavior, API responses, security, recommendation accuracy and cold-start scenarios.
Sprint	Schedule catalogue, user profile, recommendation engine, dashboard and testing work in appropriate Agile sprints.

AI. Recommendation Engine – High-Level Requirement Flow

Book Data → Data Cleaning → Feature Extraction → Recommendation Model → Candidate Books → Ranking → Personalized Recommendations → User Feedback → Model/Profile Update

Initial AI approach: Content-based filtering using book metadata and descriptions. Improvement path: combine content-based results with collaborative filtering to form a hybrid recommender.

DB. Core Data Requirements

Entity	Key Information
User	User ID, name, email, role, preferences, account status
Book	Book ID, title, description, publication details, author, genre, tags
Author	Author ID, name, biography
Genre	Genre ID, genre name
Rating	Rating ID, user ID, book ID, score, timestamp
Reading History	User ID, book ID, status, start/end dates, progress
Wishlist	User ID, book ID, created date
Recommendation	User ID, book ID, score, reason, model version, timestamp
Recommendation Feedback	User ID, recommendation ID, feedback type, timestamp

15. Requirement Analysis Checklist

- Business — What problem are we solving? What is the business objective? Who are the stakeholders?
- Users — Who will use the system? What are their roles? What can each role do?
- Functional — What should the system do? What features, recommendation behavior and business rules are required?
- Non-functional — What are the security, performance, availability, scalability, usability and privacy expectations?
- AI — What data will the model use? What recommendation method will be used? How will cold-start and feedback be handled?
- Data — What entities and relationships are required? What information must be stored and protected?
- Validation — Are the requirements clear, complete, testable and supported by acceptance criteria?
- Evaluation — How will functional correctness and AI recommendation quality be measured?

What Comes Next?

The requirements created in this module become the input for the next modules. The team will progressively move from what needs to be built to how it will be designed, modeled, tested and delivered.

